

Lab Report: Implementation of Stack in C

Student: Bal Kumar

Course/Lab: Data Structures Lab

Date: 21 December 2025

Objective

To implement the Stack abstract data type (ADT) in C using two approaches: (1) Array-based static stack, and (2) Linked-list-based dynamic stack. The lab covers core operations—push, pop, peek/top, isEmpty, isFull—along with error handling for overflow and underflow, time complexity analysis, and test cases.

Theory

A Stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle. Elements are inserted (push) and removed (pop) from the same end called the 'top'. Common applications include expression evaluation, syntax parsing, function call management (call stack), undo/redo, and backtracking.

Core Operations

Operation	Description	Average Time Complexity
push(x)	Insert element x at the top	O(1)
pop()	Remove and return the top element	O(1)
peek()/top()	Return (without removing) the top element	O(1)
isEmpty()	Check if the stack has no elements	O(1)
isFull()	(Array-only) Check if stack is at capacity	O(1)

Algorithms (High-Level)

Array-based Stack:

1. Initialize an array of fixed capacity and set top = -1. 2. push(x): if top == capacity-1 → overflow; else set top = top+1 and arr[top] = x. 3. pop(): if top == -1 → underflow; else return arr[top] and set top = top-1. 4. peek(): if top == -1 → underflow; else return arr[top]. 5. isEmpty(): return (top == -1). 6. isFull(): return (top == capacity-1).

Linked-list-based Stack:

1. Each node contains data and a pointer to the next node. Maintain a pointer 'top'. 2. push(x): allocate a new node, set node->data = x, node->next = top, update top = node. 3. pop(): if top == NULL → underflow; else temp = top; top = top->next; free(temp) and return temp->data. 4. peek(): if top == NULL → underflow; else return top->data. 5. isEmpty(): return (top == NULL).

C Implementation: Array-based Stack

```
#include <stdio.h>
```

```

#include <stdlib.h>

#define CAPACITY 5 // change capacity as needed

typedef struct {
    int arr[CAPACITY];
    int top; // index of the top element (-1 when empty)
} Stack;

void init(Stack *s) {
    s->top = -1;
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

int isFull(Stack *s) {
    return s->top == CAPACITY - 1;
}

void push(Stack *s, int x) {
    if (isFull(s)) {
        printf("Error: Stack Overflow\n");
        return;
    }
    s->arr[++(s->top)] = x; // pre-increment top and assign
    printf("Pushed %d\n", x);
}

int pop(Stack *s) {
    if (isEmpty(s)) {
        printf("Error: Stack Underflow\n");
        return -1; // sentinel value
    }
    int value = s->arr[(s->top)--]; // post-decrement top
    printf("Popped %d\n", value);
    return value;
}

int peek(Stack *s) {
    if (isEmpty(s)) {
        printf("Error: Stack is empty\n");
        return -1;
    }
    return s->arr[s->top];
}

int main() {
    Stack s;
    init(&s);

    // Test cases
    pop(&s); // underflow
    push(&s, 10);
    push(&s, 20);
    push(&s, 30);
    printf("Top = %d\n", peek(&s));
    push(&s, 40);
    push(&s, 50);
    push(&s, 60); // overflow
    while (!isEmpty(&s)) {

```

```

        pop(&s);
    }
    return 0;
}

```

C Implementation: Linked-list-based Stack

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

typedef struct {
    Node *top; // pointer to top node
} Stack;

void init(Stack *s) {
    s->top = NULL;
}

int isEmpty(Stack *s) {
    return s->top == NULL;
}

void push(Stack *s, int x) {
    Node *node = (Node*)malloc(sizeof(Node));
    if (!node) {
        printf("Error: Memory allocation failed\n");
        return;
    }
    node->data = x;
    node->next = s->top;
    s->top = node;
    printf("Pushed %d\n", x);
}

int pop(Stack *s) {
    if (isEmpty(s)) {
        printf("Error: Stack Underflow\n");
        return -1;
    }
    Node *temp = s->top;
    int value = temp->data;
    s->top = temp->next;
    free(temp);
    printf("Popped %d\n", value);
    return value;
}

int peek(Stack *s) {
    if (isEmpty(s)) {
        printf("Error: Stack is empty\n");
        return -1;
    }
    return s->top->data;
}

void clear(Stack *s) {
    while (!isEmpty(s)) {

```

```

        pop(s);
    }
}

int main() {
    Stack s;
    init(&s);

    pop(&s);           // underflow
    push(&s, 10);
    push(&s, 20);
    push(&s, 30);
    printf("Top = %d\n", peek(&s));
    clear(&s);
    return 0;
}

```

Error Handling & Edge Cases

- Overflow (array): occurs when pushing into a full stack.
- Underflow (both): occurs when popping from or peeking into an empty stack.
- Sentinel values: returning -1 for errors—alternatively, use status codes or pointers to signal failure.
- Dynamic memory (list): always check malloc() results and free nodes to prevent leaks.

Complexity Analysis

All core operations (push, pop, peek, isEmpty, isFull) run in $O(1)$ time for both implementations. The array-based stack uses $O(n)$ space with fixed maximum capacity `CAPACITY`, while the linked-list-based stack uses $O(n)$ space with per-node overhead and grows dynamically until memory exhaustion.

Sample I/O (Array-based)

```

Error: Stack Underflow
Pushed 10
Pushed 20
Pushed 30
Top = 30
Pushed 40
Pushed 50
Error: Stack Overflow
Popped 50
Popped 40
Popped 30
Popped 20
Popped 10

```

Conclusion

We successfully implemented the Stack ADT in C using both array and linked list approaches. The array-based method is simple and cache-friendly but limited by fixed capacity, whereas the linked-list-based method offers dynamic growth with additional memory overhead. Both designs achieve constant-time core operations and demonstrate proper error handling for overflow and underflow.

Viva / Oral Questions

Q1	What is the LIFO principle?
----	-----------------------------

Q2	Differentiate between array-based and linked-list-based stacks.
Q3	How do you detect overflow and underflow?
Q4	What is the time complexity of push/pop?
Q5	Real-world uses of stacks?